

Unified Broker

Implementation Report

Zerodha Kite ♦ Interactive Brokers ♦ AI Portfolio Intelligence

Prepared by: Meghna | March 2026

Python 3.13 · kiteconnect · ib_insync · Claude claude-opus-4-6

Read-only access · No order placement · Credentials held under NDA

1. Executive Summary

This report documents the complete implementation of a **unified multi-broker data layer** connecting **Zerodha Kite** (Indian equities and F&O) with **Interactive Brokers** (US and global equities). The system surfaces consolidated portfolio data through a Streamlit dashboard and an AI-powered natural-language assistant backed by **Anthropic Claude claude-opus-4-6** with native tool-use (function calling).

All nine assignment tasks are implemented as discrete, testable Python modules. The codebase is **read-only throughout**; no order-placement logic exists anywhere. A full mock-data demo mode (demo.py) allows end-to-end testing without live credentials.

Task	Title	Key Output
1	Kite Auth & Session	OAuth 2.0 login; token auto-stored in .env; session guard
2	Kite Portfolio Fetch	Holdings, overnight + intraday positions, order book → DataFrames
3	Kite WebSocket Ticks	Live tick stream → SQLite; console alerts on threshold cross
4	IBKR TWS / Gateway Auth	ib_insync connection; exponential back-off reconnect
5	IBKR Portfolio Fetch	Positions, NAV, cash, P&L; multi-currency (USD + INR)
6	IBKR Live Market Data	Mid-price every 5 s → CSV for ≥ 60 s (AAPL, MSFT, GOOGL)
7	Unified Portfolio View	Merged Kite + IBKR DataFrame; dual INR/USD totals
8	Dashboard UI	Streamlit: holdings table, tick chart, broker toggle, AI bar
9	AI Layer (Differentiator)	Claude tool-use: NL Q&A, risk scorer, regime detector, anomalies

2. Architecture & Project Structure

The project follows a layered architecture. Each broker has its own package (kite/, ibkr/) with dedicated auth, portfolio, and data-streaming modules. A broker-agnostic unified/ layer merges data and normalises currencies. The dashboard/ and ai/ layers consume the unified output.

File / Module	Responsibility
config/settings.py	Centralised config; reads all env vars; defines data paths
kite/auth.py	OAuth 2.0 handshake, token persistence, session validation
kite/portfolio.py	Holdings, net + intraday positions, order book → pandas
kite/ticks.py	KiteTicker WebSocket stream → SQLite; threshold alert engine
ibkr/auth.py	ib_insync connect, exponential back-off, disconnect handler
ibkr/portfolio.py	Portfolio positions, account summary, live FX rate
ibkr/market_data.py	Live quote subscription; mid-price → CSV every 5 s
unified/portfolio.py	Broker merge, currency normalisation, dual-currency totals
dashboard/app.py	Streamlit UI: KPIs, holdings table, charts, AI query bar
ai/assistant.py	Claude tool-use loop, risk scorer, regime detector, REPL
run.py	Single CLI entry point for all 9 tasks
demo.py	Full end-to-end mock demo — zero credentials required
.env.example	Credential template; actual .env is gitignored

Data Flow

Kite REST API and WebSocket feed into kite/portfolio.py and kite/ticks.py. IBKR TWS/Gateway feeds into ibkr/portfolio.py and ibkr/market_data.py. Both streams converge in unified/portfolio.py, which applies live FX conversion and produces a single normalised DataFrame consumed by the dashboard and AI layer.

3. Task-by-Task Implementation Detail

Task 1 — Kite Auth & Session (kite/auth.py)

Implements the **Zerodha Kite Connect OAuth 2.0 flow**. On first run, the user is directed to the Kite login URL; after authentication, Kite redirects with a `request_token` which is exchanged for a long-lived `access_token`. The token is persisted to `.env` via `python-dotenv`'s `set_key()` so subsequent runs bypass the browser step.

Session guard: before reusing a stored token, `_is_session_valid()` pings the `/profile` endpoint. On failure the full OAuth flow is re-triggered automatically — satisfying the auto re-auth requirement. A CLI `invalidate_session()` helper clears the token.

Mechanism	Detail
Library	<code>kiteconnect >= 5.0.1</code>
Token storage	<code>python-dotenv set_key()</code> → <code>.env</code> file
Session check	<code>kite.profile()</code> ping — raises on expiry
Re-auth trigger	Automatic on any session-check failure
Output	KiteConnect object with active <code>access_token</code>

Task 2 — Kite Portfolio Fetch (kite/portfolio.py)

Three functions cover the full portfolio picture: **`fetch_holdings()`** pulls the equity holdings book (`kite.holdings()`); **`fetch_positions()`** returns a tuple of overnight (*net*) and intraday (*day*) positions; **`fetch_orders()`** returns today's order book. All three normalise broker field names (e.g. `tradingsymbol` → `symbol`) and compute derived columns (`market_value = qty × ltp`, `cost_basis = qty × avg_cost`).

The **`portfolio_summary()`** helper renders rich-formatted tables to stdout with colour-coded P&L totals. Every function is independently importable and accepts a KiteConnect object, making them composable from the unified layer and dashboard without duplication.

Function	API call	DataFrame columns
<code>fetch_holdings()</code>	<code>kite.holdings()</code>	<code>symbol</code> , <code>exchange</code> , <code>qty</code> , <code>avg_cost</code> , <code>ltp</code> , <code>unrealised_pnl</code> , <code>day_change</code> , <code>market_value</code>
<code>fetch_positions()</code>	<code>kite.positions()</code>	<code>symbol</code> , <code>exchange</code> , <code>product</code> , <code>qty</code> , <code>avg_cost</code> , <code>ltp</code> , <code>unrealised_pnl</code> , <code>market_value</code> (>
<code>fetch_orders()</code>	<code>kite.orders()</code>	<code>order_id</code> , <code>symbol</code> , <code>side</code> , <code>order_type</code> , <code>qty</code> , <code>price</code> , <code>status</code> , <code>placed_at</code>

Task 3 — Kite WebSocket Ticks (kite/ticks.py)

Uses **KiteTicker** in threaded mode to stream **full-quote ticks** (LTP, bid, ask, volume) for ITC (token 424961), ONGC (633601), and NIFTY 50 (256265). Each tick is persisted to a local **SQLite database** (`data/ticks.db`) with a UTC ISO-8601 timestamp, instrument token, symbol, LTP, volume, bid, and ask.

Alert engine: `_check_alerts()` compares each LTP against configured thresholds from the environment (ITC_ALERT_ABOVE, ONGC_ALERT_BELOW). It fires once per crossing and auto-resets when the price returns to the safe side — preventing alert floods. Alerts are printed in ANSI yellow/red and written to the log.

Lifecycle: the stream handles `on_connect`, `on_close`, `on_error`, and `on_reconnect` callbacks. A SIGINT handler triggers a `clean kws.stop()`. An optional `duration_seconds` parameter stops the stream after a fixed interval (used by the test runner).

Task 4 — IBKR TWS / Gateway Auth (`ibkr/auth.py`)

Connects to a live TWS or IB Gateway instance using `ib_insync`. The `get_ibkr_client()` function retries up to 5 times with **exponential back-off** (base 5 s, cap 60 s) before raising a descriptive error.

Two event handlers are attached at connect time. **disconnectedEvent** triggers an automatic reconnect attempt after a 5-second delay, simulating the survivability required by the spec. **errorEvent** silently ignores IBKR informational codes (2104, 2106, 2119, etc.) while logging true errors at ERROR level. After connecting, `_log_account_info()` logs the managed account IDs and NAV to confirm the session is live.

Feature	Implementation
Library	<code>ib_insync >= 0.9.86</code>
Ports	7497 = TWS paper 7496 = TWS live 4001 = IB Gateway
Retry strategy	5 attempts, exponential back-off (5 → 10 → 20 → 40 → 60 s)
Auto-reconnect	<code>ib.disconnectedEvent</code> callback; retries after 5 s
Informational codes	2104, 2106, 2107, 2108, 2119, 2157, 2158 suppressed at DEBUG
Reconnect test	Explicit <code>ib.disconnect()</code> + <code>get_ibkr_client()</code> in CLI entry-point

Task 5 — IBKR Portfolio Fetch (`ibkr/portfolio.py`)

fetch_positions() iterates `ib.portfolio()` and normalises each position into a row containing broker, symbol, secType, exchange, currency, qty, avg_cost, mkt_price, mkt_value, unrealised_pnl, and an **INR equivalent** column. The live USD/INR mid-rate is fetched first from IBKR's own FX market data (`Forex('USDINR')`); if unavailable, the env-var fallback is used.

Multi-currency handling: USD positions are multiplied by the FX rate; INR-denominated positions are passed through unchanged; other currencies are treated as USD with a log warning. **fetch_account_summary()** queries `NetLiquidation`, `TotalCashValue`, `UnrealizedPnL`, `RealizedPnL`, `GrossPositionValue`, and `MaintMarginReq` across all managed accounts.

Column	Source	Notes
broker	hardcoded 'IBKR'	—
symbol	<code>pos.contract.symbol</code>	—
currency	<code>pos.contract.currency</code>	USD / INR / other
qty	<code>pos.position</code>	negative = short
avg_cost	<code>pos.averageCost</code>	rounded 4 dp
mkt_value	<code>pos.marketValue</code>	native currency
unrealised_pnl	<code>pos.unrealizedPNL</code>	native currency

Column	Source	Notes
inr_equiv	mkt_value × usd_inr	None for exotic CCY

Task 6 — IBKR Live Market Data (ibkr/market_data.py)

stream_market_data() subscribes to live SMART-routed quotes for AAPL, MSFT, and GOOGL using `ib.reqMktData()`. Every 5 seconds it samples bid, ask, and computed mid-price $((\text{bid} + \text{ask}) / 2)$ from each ticker and appends a row to a CSV file with a UTC ISO-8601 timestamp. The stream runs for at least 60 seconds (configurable) and cancels all market-data subscriptions on exit.

A SIGINT handler sets a stop flag so the loop exits cleanly. The CSV header is written only if the file is new or empty, allowing multiple runs to append without duplication. The output path is configurable and defaults to `logs/ibkr_midprice.csv`.

Task 7 — Unified Portfolio View (unified/portfolio.py)

build_unified_portfolio() accepts DataFrames from all four sources (Kite holdings, Kite net positions, IBKR positions) and an optional USD/INR rate, then produces a single normalised DataFrame with consistent columns across both brokers.

Currency normalisation: Kite holdings are always INR; IBKR USD positions are converted by multiplying market value and P&L by the live FX rate. All rows carry both `market_value_inr` / `market_value_usd` and `unrealised_pnl_inr` / `unrealised_pnl_usd` — enabling single-column aggregation in either currency.

FX rate: fetched from `api.exchangerate-api.com` at runtime with a 5-second timeout; falls back to the `USD_INR_RATE` env var on failure. **print_unified_summary()** renders a rich-formatted table with broker subtotals and grand totals in both currencies.

Output Column	Type	Description
broker	str	'Kite' or 'IBKR'
symbol	str	Ticker / trading symbol
qty	int	Quantity held (negative = short)
currency	str	Native currency of the position
avg_cost	float	Average acquisition cost in native CCY
ltp	float	Last traded / market price
market_value_inr	float	Market value converted to INR
market_value_usd	float	Market value converted to USD
unrealised_pnl_inr	float	Unrealised P&L in INR
unrealised_pnl_usd	float	Unrealised P&L in USD

Task 8 — Dashboard UI (dashboard/app.py)

A **Streamlit** application providing a live, read-only view of the unified portfolio. Data is fetched with a 30-second cache TTL (`@st.cache_data`) to balance freshness against API rate limits.

Component	Detail
KPI cards (4)	Total Value (■), Total Value (\$), Unrealised P&L (■), Open Positions
Holdings table	Colour-coded P&L (green/red), formatted currencies, broker-filterable
Allocation pie chart	Plotly pie: portfolio weight by broker in INR
Kite tick chart	Plotly line chart of LTP over time from SQLite; symbol selector
IBKR mid-price chart	Plotly line chart from CSV; symbol selector
Order book	Kite order book in collapsible expander
AI query bar	Text input + preset questions; calls Claude via <code>run_ai_query()</code>
Broker toggle	Sidebar multiselect filters holdings table to Kite / IBKR / Both
Auto-refresh	Sidebar checkbox: clears cache and <code>st.rerun()</code> every 30 s

Graceful degradation: each broker's data loader is wrapped in a `try/except`; if Kite or IBKR is unavailable, a sidebar warning is shown and the remaining data still renders. The dashboard works with mock data from `demo.py` even without any live credentials.

Task 9 — AI Layer (ai/assistant.py)

The AI layer is built on **Anthropic Claude claude-opus-4-6** with **native tool-use (function calling)**. Rather than pasting a static portfolio JSON into the system prompt, Claude is given a set of callable tools and decides autonomously which to invoke — and in what order — before composing a grounded answer.

3.9.1 Tool-Use Architecture

The `ask()` function runs an **agentic loop** (up to 8 rounds). On each round, Claude's response is inspected: if `stop_reason == 'tool_use'`, every `tool_use` block is dispatched to the corresponding Python function, results are JSON-serialised and returned as `tool_result` messages, and the loop continues. When `stop_reason == 'end_turn'`, the final text is returned.

Tool	Returns
<code>get_positions(broker)</code>	All positions; optionally filtered to Kite or IBKR
<code>get_pnl_summary()</code>	Total INR/USD P&L, market value, top gainer, top loser, by-broker split
<code>get_sector_exposure()</code>	Market value and % weight per sector (Technology, Banking, Energy...)
<code>get_risk_score()</code>	0-100 risk score + label + HHI components + drawdown metrics
<code>get_tick_anomalies()</code>	Z-score scan of SQLite DB; SPIKE ▲ / DIP ▼ flagged symbols
<code>get_market_regime()</code>	TRENDING / RANGING / HIGH-VOLATILITY from tick volatility stats

3.9.2 Portfolio Risk Scorer

A quantitative **0-100 composite risk score** built from five signals:

Signal	Weight	Method
Position concentration (HHI)	35%	Herfindahl-Hirschman Index across individual holdings
Sector concentration (HHI)	25%	HHI across sector buckets (Technology, Banking, Energy...)
Broker split	20%	HHI across Kite vs IBKR allocation
Largest single drawdown	10%	Max unrealised loss / total portfolio value × 100
Total drawdown ratio	10%	Sum of all unrealised losses / total portfolio value × 100

Score 0-24 = **LOW** · 25-49 = **MODERATE** · 50-74 = **HIGH** · 75-100 = **VERY HIGH**. Claude uses this score to contextualise answers about risk without guessing.

3.9.3 Market Regime Detector

Reads the most recent ticks from SQLite and, per symbol, computes: (a) the **rolling z-score** of LTP and takes the median absolute value as a volatility proxy; (b) a **linear trend slope** via `numpy.polyfit`. Regime classification:

- **HIGH-VOLATILITY** — average median $|z| > 1.5$ across all symbols
- **TRENDING** — average $|\text{slope}| > 0.05$ (consistent directional drift)
- **RANGING** — everything else (low dispersion, no clear direction)

The regime is surfaced in the daily brief and available as a standalone tool call so Claude can contextualise P&L moves ('markets were high-vol today, which explains the wide swings in your NIFTY 50 position').

3.9.4 Daily Brief, Risk Report & Anomaly Explanation

Feature	Prompt strategy	Claude actions
Daily brief	Structured 6-8 bullet prompt covering val, P&L, sector, regime, get_sector_exposure, get_risk_score, get_risk_observation	
Risk report	Plain-English risk breakdown: score, drivers, get_risk_score, get_sector_exposure; synthesises narrative	
Anomaly explain	Anomaly table passed in message; Claude asks questions - reference flagged symbols are held; explains re	

3.9.5 Multi-Turn Conversational REPL

The ask() function accepts and returns a history: list[dict] parameter containing the full Anthropic messages array. The REPL passes this history back on every turn, giving Claude **full conversation memory** within a session. This enables follow-up questions like 'which stocks are driving that?' after asking about risk — without re-explaining context.

CLI flag	Behaviour
--ask 'question'	One-shot answer with tool-use
--brief	Plain-English daily brief (6-8 bullets)
--risk	Detailed risk report with component breakdown
--anomaly	Z-score scan + Claude explains each anomalous symbol
(no flag)	Interactive multi-turn REPL with conversation memory

4. Setup & Running

4.1 Installation

```
git clone <repo-url> && cd unified-broker
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # then fill in credentials
```

4.2 Without API Keys (Mock Demo)

Run the full end-to-end demonstration with synthetic data — no credentials needed:

```
python demo.py # all 9 tasks
python demo.py --task 7 # single task
```

4.3 With Live Credentials

Command	Task
python run.py --task 1	Kite OAuth login
python run.py --task 2	Live Kite portfolio
python run.py --task 3 --duration 60	Tick stream (60 s)
python run.py --task 4	IBKR connect + reconnect test
python run.py --task 5	IBKR live portfolio
python run.py --task 6 --duration 60	IBKR mid-prices → CSV
python run.py --task 7	Unified portfolio
streamlit run dashboard/app.py	Dashboard (http://localhost:8501)
python run.py --task 9 --brief	AI daily brief
python run.py --task 9 --risk	AI risk report
python run.py --task 9 --anomaly	AI anomaly scan
python run.py --task 9 --ask "..."	One-shot AI question
python run.py --task 9	Multi-turn AI REPL

5. Dependencies

Package	Version	Purpose
kiteconnect	≥ 5.0.1	Zerodha Kite REST API + KiteTicker WebSocket
ib_insync	≥ 0.9.86	Interactive Brokers TWS / Gateway client
pandas	≥ 2.0.0	DataFrame normalisation across all tasks
numpy	≥ 1.24.0	Numerical operations (HHI, z-score, polyfit)
anthropic	≥ 0.25.0	Claude API client with tool-use support
streamlit	≥ 1.32.0	Dashboard UI framework
plotly	≥ 5.18.0	Interactive charts (tick, mid-price, pie)
python-dotenv	≥ 1.0.0	Env var loading + set_key() for token persistence
requests	≥ 2.31.0	Live FX rate fetch
sqlalchemy	≥ 2.0.0	SQLite ORM (tick DB)
aiohttp	≥ 3.9.0	Async HTTP (ib_insync dependency)
rich	≥ 13.7.0	Terminal table rendering and colour output
tabulate	≥ 0.9.0	Markdown table formatting for Claude prompts

6. Security & Compliance

- `.env` is gitignored — credentials are never committed to version control.
- `data/` and `logs/` directories are gitignored — tick DBs and CSVs stay local.
- All broker access is **read-only**. No order-placement code exists anywhere in the codebase.
- IBKR informational error codes are filtered to prevent false-positive alerting.
- The Anthropic API key is validated at runtime; the AI layer raises a clear error if absent rather than silently failing.
- Credentials shared under NDA must not be forwarded or embedded in any committed file.

7. What I'd Build Next

7.1 Infrastructure

- **TimescaleDB tick store** — replace SQLite with TimescaleDB for efficient continuous aggregates, multi-day VWAP queries, and month-over-month regime analysis without full-table scans.
- **Position reconciliation job** — nightly diff between broker-reported positions and locally cached state to detect sync drift, missed fills, or corporate actions.
- **Options Greeks dashboard** — pull IV, delta, gamma, theta for Kite F&O; positions and surface a live Greeks table with expiry countdown timers.
- **Streamlit WebSocket push** — replace poll-based cache TTL with a true WebSocket push so holdings update on every tick without a full page reload.

7.2 AI Differentiators

- **Anomaly-to-news pipeline** — when `get_tick_anomalies` fires, automatically call a web-search tool, attach the top 3 relevant headlines to the Claude context, and explain the price move with cited sources.
- **Voice morning brief** — pipe `daily_brief()` output through a TTS API (ElevenLabs or AWS Polly) and deliver as an audio notification before market open.
- **NL order preview** (read-only) — convert natural language ('show me what 10 more AAPL shares would do to my portfolio') into a structured preview showing new weight, new risk score delta, and estimated cost — displayed for review, never submitted.
- **Backtester integration** — feed tick history from TimescaleDB into a vectorbt backtest triggered by a plain-English strategy description and surface the equity curve inside the dashboard.
- **Conversational portfolio journal** — after each REPL session, auto-summarise the conversation into a dated entry; Claude can later answer 'how has my risk score trended over the last month?' using its own historical outputs as context.

8. Sample AI Outputs

8.1 Natural-Language Q&A;

Q: What is my total unrealised P&L in USD today?

A (Claude): Your combined unrealised P&L is **■8,202.74 / \$98.23** — **■6,313.50** from Kite positions (ITC and TCS leading) and **■1,889.24** from your IBKR book (AAPL +\$82.10, MSFT +\$86.34, GOOGL +\$24.82). The only drag is HDFCBANK at **-■339**.

8.2 Risk Report

Prompt: python run.py --task 9 --risk

A (Claude): Risk score: **31/100 (MODERATE)**. Your portfolio is reasonably diversified across 8 positions (HHI = 18.4) but carries meaningful sector concentration — Technology accounts for 55% of total value (HHI = 42.1). Broker split is skewed: 78% is held at Kite vs 22% at IBKR (HHI = 61.0). The only unrealised loss is HDFCBANK (**-■339**, 0.4% of portfolio), so drawdown risk is minimal. To reduce the top risk factor, consider diversifying the Technology overweight into other sectors such as Energy or FMCG.

8.3 Tick Anomaly Detection

symbol	latest_ltp	mean_ltp	std_ltp	z_score	direction
ITC	520.00	450.31	5.0384	13.831	SPIKE ▲

Claude's analysis: ITC's latest tick of **■520** is 13.8 standard deviations above the rolling mean of **■450.31** — an extreme outlier almost certainly representing a data-feed error or circuit-breaker event rather than genuine price discovery. You hold 100 shares of ITC (Kite), so if genuine, your unrealised P&L would increase by approximately **■5,785**. Recommend waiting for the next 2-3 ticks to confirm before drawing conclusions.

8.4 Daily Brief

- **Total portfolio value:** **■8,97,312 / \$10,747** (USD/INR = 83.50).
- **Unrealised P&L:** **+■8,202.74 / +\$98.23** — portfolio is in the green overall.
- **Top gainer:** ITC **+■3,215** (100 shares, avg **■430** vs LTP **■462.15**).
- **Top loser:** HDFCBANK **-■339** (15 shares, avg **■1,610** vs LTP **■1,587.40**).
- **Sector overweight:** Technology at 55% of total value — TCS and INFY are the main drivers.
- **Market regime:** RANGING — tick data shows low z-score dispersion and no clear directional trend.
- **Risk score:** 31/100 (MODERATE) — sector concentration is the top driver.
- **Observation:** IBKR book is all-green (+\$193.26 combined); US tech positions benefiting from overnight gains.